

O texto *Design by Contract*, escrito por Bertrand Meyer, fala sobre uma forma de desenvolver software de maneira mais organizada e confiável. O autor compara o funcionamento de programas com contratos da vida real, mostrando que em um sistema cada parte deve ter responsabilidades bem definidas. A ideia principal é que os componentes do software façam “acordos” entre si, deixando claro o que cada um deve fazer e o que pode esperar do outro.

Segundo o autor, durante o desenvolvimento de um programa é comum dividir tarefas em partes menores. Algumas dessas tarefas podem ser feitas diretamente pelo programador e outras podem ser delegadas para rotinas ou métodos separados. Quando isso acontece, é necessário criar regras claras para que tudo funcione corretamente. Essas regras formam o chamado contrato de software.

O Design by Contract trabalha principalmente com três conceitos: pré-condições, pós-condições e invariantes. As pré-condições representam aquilo que deve ser verdadeiro antes da execução de uma rotina. Ou seja, são regras que o cliente do método deve cumprir antes de utilizá-lo. Se essas condições não forem respeitadas, o método não é obrigado a funcionar corretamente. Já as pós-condições mostram aquilo que a rotina garante após sua execução. Elas definem o resultado esperado depois que o método termina.

O texto apresenta o exemplo de uma tabela que recebe novos elementos. Antes da inserção, a tabela não pode estar cheia, pois isso é uma pré-condição. Depois da execução, a pós-condição garante que o elemento foi inserido corretamente e que a quantidade de itens aumentou. Dessa forma, tanto quem utiliza quanto quem desenvolve a rotina sabem exatamente quais são suas responsabilidades.

Outro conceito importante explicado no texto é o de invariantes de classe. Os invariantes são condições que devem permanecer verdadeiras durante toda a existência de um objeto. Por exemplo, a quantidade de elementos de uma estrutura nunca pode ser negativa e também não pode ultrapassar sua capacidade máxima. Essas regras ajudam a manter a consistência do sistema e evitam erros durante a execução do programa.

Bertrand Meyer também critica a chamada programação defensiva. Nesse modelo tradicional, os programadores fazem muitas verificações repetidas para evitar erros. O autor acredita que isso deixa o código mais complexo e difícil de manter. No Design by Contract, cada responsabilidade deve pertencer somente a uma parte do sistema. Assim, evita-se repetição de verificações e o código fica mais simples e organizado.

Além disso, o texto fala sobre assertions, que são expressões booleanas usadas para verificar se as regras do contrato estão sendo cumpridas. Elas ajudam tanto na documentação quanto na

identificação de erros no software. Quando uma assertion falha, significa que existe algum problema no programa, seja de implementação ou de uso incorreto de uma rotina.

Outro assunto abordado é o tratamento de exceções. O autor defende que exceções devem ser utilizadas apenas em situações realmente anormais, como falta de memória, erros de hardware ou falhas inesperadas. Para ele, muitos problemas podem ser evitados apenas com contratos bem definidos, sem necessidade de tratamentos excessivos de erro.

Concluindo, o Design by Contract é uma metodologia que ajuda a criar softwares mais seguros, organizados e confiáveis. Através de contratos claros entre as partes do sistema, é possível reduzir erros, melhorar a manutenção do código e facilitar o entendimento do funcionamento do programa. O texto mostra que definir corretamente as responsabilidades de cada componente é fundamental para o desenvolvimento de sistemas de qualidade.